technica engineering

Member of KPIT Group

#OneStepAhead

Remote API

Technica Products



version: 1.4

Release date: 01.04.2026

Table of content

1 Introduction 5
1.1 Validity 5
1.2 Scope of this document 5
1.3 Glossary 5
2 Remote Control API, General Concepts 7
2.1 Documentation Format 7
2.2 Port coding 7
3 Application HTTP Get Methods 9
3.1 [Get] Retrieve the device status in binary format 9
3.2 [Get] Retrieve the current configuration in json format 9
3.3 [Get] Retrieve the current configuration in msgpack format 9
3.4 [Get] Retrieve the default configuration in json format 9
3.5 [Get] Retrieve the default configuration in msgpack format 9
3.6 [Get] Retrieve the equipment information in json format 10
3.7 [Get] Retrieve the equipment information in msgpack format 10
3.8 [Get] Retrieve the status information in json format 10
3.9 [Get] Retrieve the status information in msgpack format 10
3.10 [Get] Retrieve the LLDP neighbor list in json format 10
3.11 [Get] Retrieve the LLDP interfaces list in json format 11
3.12 [Get] Retrieve the Address Resolution Table in json format 11
3.13 [Get] Retrieve the Address Resolution Table in msgpack format 11
3.14 [Get] Retrieve a MKA key in json format 11
3.15 [Get] Retrieve a MKA key in msgpack format 11
3.16 [Get] Retrieve the port segmentation in json format 11
3.17 [Get] Retrieve the port segmentation in msgpack format 12
3.18 [Get] Retrieve the TC10 State in json Format 12
4 Application HTTP Post Methods 13
4.1 [Post] Validate configuration 13
4.2 [Post] Configuration Control 13
4.2.1 [Post][Save] Write the whole configuration in a persistent way 13
4.2.2 [Post][Save] Write part of the configuration in a persistent way 13
4.2.3 [Post][Save] Write a whole configuration in a non-persistent way 13
4.2.4 [Post][Save] Write a configuration in a non-persistent way 14
4.3 [Post] Action commands 14
4.3.1 [Post][Action] Save the whole configuration in a persistent way 14

4.3.2 [Post][Action] Cold reset 14
4.3.3 [Post][Action] Warm reset 14
4.3.4 [Post][Action] Alive command 15
4.3.5 [Post][Action] Jump to bootloader mode 15
4.3.6 [Post][Action] Force sleep mode 15
4.3.7 [Post][Action] Clear statistics 15
4.4 [Post] On-the-fly commands 15
4.4.1 [Post][OnTheFly] Address Resolution Table 15
4.4.1.1 [Post][OnTheFly][ART] Remove all automatic learned entries 15
4.4.1.2 [Post][OnTheFly][ART] Remove an entry 16
4.4.1.3 [Post][OnTheFly][ART] Add a static entry 16
4.4.1.4 [Post][OnTheFly][ART] Replace an entry 16
4.4.2 [Post][OnTheFly] Set port physical layer parameters 17
4.4.3 [Post][OnTheFly][Ptp] Set time on PTP grandmaster partitions 18
4.4.4 [Post][OnTheFly] Modify MKA key 19
4.4.5 [Post][OnTheFly] Set Port Segmentation 19
4.4.6 [Post][OnTheFly] Set The VLAN configuration per port 20
4.4.7 [Post][OnTheFly] Advanced Filter 21
4.4.8 [Post][OnTheFly] Set system date and time 22
4.4.9 [Post][OnTheFly] Trigger remote I2C operation 23
4.4.10 [Post][OnTheFly] Start Frame Generator 23
4.4.11 [Post][OnTheFly] Stop Frame Generator 24
4.4.12 [Post][OnTheFly] Start IEEE Test Mode 25
4.4.13 [Post][OnTheFly] Stop IEEE Test Mode 25
4.4.14 [Post][OnTheFly] Cable Diagnostics 26
4.4.15 [Post][OnTheFly] Set a TC10 Local Sleep Event 26
4.4.16 [Post][OnTheFly] Set a TC10 Local Wake Event 27
4.4.17 [Post][OnTheFly] Set a TC10 Remote Wake-Up (WUP) Event 27
4.5 [Post] Password commands 27
4.5.1 [Post][Password] set password 28
4.5.2 [Post][Password] clear password 28
4.5.3 [Post][Password] nonce command 29
4.5.4 [Post][Password] protected command 29
5 Bootloader API 30
5.1 [Post] Return back to main application 30
5.2 [Post] Retrieve the system information in json format 30
5.3 [Post] Retrieve the status of the firmware update in json format 30

5.3.1 [POST][Bootloader] Update firmware or license file 31
5.3.2 [POST][Bootloader] Save firmware or license file without reset 31
5.4 [Post] Restore configuration to factory default 31
6 Examples of use 32
6.1 Curl 32
6.1.1 [Curl] Read all current configuration data in msgpack and save it in a file 32
6.1.2 [Curl] Read all current configuration data in Json format and save it in a file 32
6.1.3 [Curl] Read part of current configuration data in Json format and save it in a file 32
6.1.4 [Curl] Read all current status data in Json format and save it in a file 32
6.1.5 [Curl] Check is HTTP requests can be accepted 32
6.1.6 [Curl] Send the complete configuration and save it in the device's NVM 32
6.1.7 [Curl] Send the complete configuration and save it in the device's RAM 33
6.1.8 [Curl] Validate a configuration 33
6.1.9 [Curl] Jump to bootloader 33
6.1.10 [Curl] Update the firmware once in bootloader 33
6.2 Python snippet scripts 33
7 Remote API reference code 35
7.1 Requirements 35
7.2 WebInterface base class definition 35
7.3 Device-Specific class definition 37
7.4 Example script application definition 37
8 Document Changelog 40

1 Introduction

If you have any questions or need further clarification, please contact our customer support at support@technica-enginnering.de

1.1 Validity

This document applies to the Technica Devices listed below, along with the hardware and software specified in the table.

Technica Device	Product Version	Firmware Version
TE-1170, TE-1171, TE-1173, TE-1175	Up to v3.3	R021.00x.200
TE-1176, TE-1177	Up to v5.4	R021.00x.200
TE-1180, TE1181	Up to v4.0	R.006.004
TE-1190	Up to v2.2	R.010.002
TE-1191	Up to v3.0	R.010.000
TE-1200	Up to v3.1	R.006.002
TE-2000, TE-2001	Up to v3.2	R.016.002
TE-2002	Up to v2.1	R.016.002
TE-2003, TE-2004	Up to v3.3	R.016.002
TE-2005	Up to v1.4	R.016.002
TE-2006	Up to v1.0	R.016.002

Ensure that this document corresponds to the Technica device you are using and the firmware version installed. If your device is running an outdated software version, please update it to the latest version before proceeding.
The User Manual is valid from the time of publishing to the publication of a new version of this User Manual.

The latest firmware version in the Customer portal
The firmware and device documentation are continuously updated. The latest documentation may differ or include additional information. Always download the most recent version from Technica Engineering's Customer Portal at https://portal.technica-engineering.de/downloads.

1.2 Scope of this document

This document describes the Remote Application Programming Interface (API). All commands utilize standard HTTP requests to retrieve data from or send data to the device.

1.3 Glossary

- URL (Uniform Resource Locator): A reference or address used to access resources on the internet, specifying the location of a resource along with the protocol used to retrieve it (e.g., HTTP, HTTPS). Learn more about URLs
- HTTP (Hypertext Transfer Protocol): A protocol used for transferring data over the web, allowing communication between clients (like browsers) and servers. It defines how messages are formatted and transmitted, and how servers respond to requests. Learn more about HTTP
    - HTTP GET Request: Used to retrieve data from a server. It is a read-only request where which data to read is included in the URL, typically used for requesting resources or information (e.g., fetching a webpage or API data).

  - HTTP POST Request: Used to send data to a server to create or update a resource. The retrieved data is included in the body of the request, not the URL. POST requests are used for actions such as submitting forms, logging in, or uploading files.

- JSON (JavaScript Object Notation): A lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate. Official JSON Documentation
- msgpack (MessagePack): A binary serialization format that efficiently encodes data, offering faster processing and smaller size compared to JSON. Official MessagePack Documentation
- NVM (Non-Volatile Memory): A type of computer memory that retains stored data even when the power is turned off. Unlike volatile memory (such as RAM), NVM does not lose its content when the device is powered down
- nonce (short for "number used once") is a unique, random value that is generated for a single use in a security process. It helps prevent replay attacks, where an attacker intercepts and reuses a valid authentication request to gain unauthorized access.

2 Remote Control API, General Concepts

Technica devices feature a remote-control API based on the HTTP protocol, offering two types of HTTP requests:

- GET Requests: Used to retrieve information from the device. Both JSON and msgpack formats are supported. JSON-formatted requests can include a PATH at the end to fetch information from a specific node, rather than retrieving all data from the root node.
- POST Requests: Used to modify the configuration and trigger actions. These requests support only the msgpack format.

2.1 Documentation Format

Each HTTP request is defined by its request method (GET or POST) and a unique URL. The URL consists of a base common to all requests, which includes the IP address assigned to the device, followed by an endpoint specific to each request.

For example, for a device with an IPv4 address of 10.0.0.200, the common `{BASE}` URL is:

```
http://10.0.0.200/
```

An example of an endpoint might be:

```
api/get/config.json
```

Combining these, the full URL would be:

```
http://10.0.0.200/api/get/config.json
```

2.2 Port coding

Some Remote API endpoints use a port index to reference a port. The port index assignment varies based on the port distribution of each Technica device. Below is the port index mapping for Technica devices:

Capture Module 100 HIGH TC10 (TE-1172) port coding

- Main ports (100BAS-T1-1 through 10BAS-T1-6) are coded as 0 through 5

Capture Module 10BASE-T1S (TE-1181) port coding

- Main ports (10BAS-T1S-1 through 10BAS-T1S-6) are coded as 0 through 5

Capture Module SerDes GMSL2/3 (TE-1190) port coding

- Main ports (GMSL LINK-1 through GMSL LINK-4) are coded as 0 through 3

Capture Module MultiGigabit (TE-1191) port coding

- Main ports (BASE-T1 LINK-1 through BASE-T1 LINK-2) are coded as 0 through 1

Network Interfacer 10BASE-T1S (TE-1200) port coding

- Main ports (10BASE-T1S-1 through 10BASE-T1S-3) are coded as 0 through 2
- SFP+ is coded as 3
- USB port is coded as 4
- Host/Control port is coded as 5
- BASE-T1 port is coded as 6

Enhanced Ethernet Switch (TE-2000 to TE-2006) port coding

- Main ports (BASE-T1-1 through BASE-T1-8) are coded as 0 through 7
- SFP+A is coded as 8 and SFP+B is coded as 9
- Host/Control port is coded as 10

3 Application HTTP Get Methods

HTTP GET methods allow to retrieve the configuration and status information in both JSON and msgpack formats.

3.1 [Get] Retrieve the device status in binary format

This command is supported on the following Technica devices:			
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177		

The `{BASE}/api/control` method returns the device status

3.2 [Get] Retrieve the current configuration in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The configuration contains all the options that users can adjust on the device. The information is organized in a hierarchical structure, grouping related configuration parameters within the same node for easier navigation and management.

The `{BASE}/api/get/config` and `{BASE}/api/get/config.json` methods return the complete configuration encoded in json format.

The `{BASE}/api/get/config/PATH` and `{BASE}/api/get/config.json/PATH` methods return only the configuration contained in the specified node by PATH, encoded in json format.

3.3 [Get] Retrieve the current configuration in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/get/config.msgpack` method returns the complete configuration encoded in msgpack format.

3.4 [Get] Retrieve the default configuration in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

A default configuration can be accessed without altering the current settings. This feature allows users to use the default configuration as a baseline for creating a new configuration, serving as an effective starting point for customization.

The `{BASE}/api/get/default_config` and `BASE/api/get/default_config.json` methods return a complete default configuration encoded in json format.

The `{BASE}/api/get/default_config/PATH` and `BASE/api/get/default_config.json/PATH` methods return only the default configuration of the specified node by PATH, encoded in json format.

3.5 [Get] Retrieve the default configuration in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/get/default_config.msgpack` method returns a complete default configuration encoded in msgpack format.

3.6 [Get] Retrieve the equipment information in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

Equipment information provides detailed internal data about the device, including hardware and software versions, network configuration, and license details.

The `{BASE}/api/get/equipment` and `{BASE}/api/get/equipment.json` methods return the complete equipment information encoded in json format.

The `{BASE}/api/get/equipment/PATH` and `{BASE}/api/get/equipment.json/PATH` methods return only the equipment information of the specified node by PATH, encoded in json format.

3.7 [Get] Retrieve the equipment information in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/get/equipment.msgpack` methods return the complete equipment information encoded in msgpack format.

3.8 [Get] Retrieve the status information in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The device gathers extensive data, such as link status and traffic statistics for each port. This information can be accessed using the status method, providing valuable insights into the device's operation and performance.

The `{BASE}/api/get/status` and `{BASE}/api/get/status.json` methods return the complete current status encoded in json format.

The `{BASE}/api/get/status/PATH` and `{BASE}/api/get/status.json/PATH` methods return only the status information of the specified node by PATH, encoded in json format.

3.9 [Get] Retrieve the status information in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/get/status.msgpack` method returns the complete status information encoded in msgpack format.

3.10 [Get] Retrieve the LLDP neighbor list in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1191	TE-1180, TE1181	TE-1172	

The `{BASE}/api/get/tdp/neighbors` and `{BASE}/api/get/tdp/neighbors.json` methods return all the devices that LLDP protocol has found encoded in json format.

The `{BASE}/api/get/tdp/neighbors/PATH` and `{BASE}/api/get/tdp/neighbors.json/PATH` methods return only the LLDP information of the specified node by PATH, encoded in json format.

3.11 [Get] Retrieve the LLDP interfaces list in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1191	TE-1180, TE1181		

The `{BASE}/api/get/tdp/interfaces` and `{BASE}/api/get/tdp/interfaces.json` methods return the information published on each port by LLDP.

The `{BASE}/api/get/tdp/interfaces/PATH` and `{BASE}/api/get/tdp/interfaces.json/PATH` methods return only the LLDP information of the specified node by PATH, encoded in json format.

3.12 [Get] Retrieve the Address Resolution Table in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/read_first_learned_entries` and `{BASE}/api/at/read_first_learned_entries.json` methods return all the MAC addresses that the switch has learned encoded in json format.

The `{BASE}/api/at/read_first_learned_entries/PATH` method returns only the Address Resolution Table information of the specified node by PATH, encoded in json format.

3.13 [Get] Retrieve the Address Resolution Table in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/read_first_learned_entries.msgpack` method returns all the MAC addresses that the switch has learned encoded in msgpack format.

3.14 [Get] Retrieve a MKA key in json format

This command is supported on the following Technica devices:				
TE-2003 to TE-2005				

The `{BASE}/api/mka/{X}/get_key` and `{BASE}/api/mka/{X}/get_key.json` methods return the parameters of the MKA key with index `{X}` encoded in json format. This endpoint provides the same format as the Post endpoint described in the chapter [Post][OnTheFly] Modify MKA key.

3.15 [Get] Retrieve a MKA key in msgpack format

This command is supported on the following Technica devices:				
TE-2003 to TE-2005				

The `{BASE}/api/mka/{X}/get_key.msgpack` method returns the parameters of the MKA key with index `{X}` encoded in json format. This endpoint provides the same format as the Post endpoint described in the chapter [Post][OnTheFly] Modify MKA key.

3.16 [Get] Retrieve the port segmentation in json format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/get/port_segmentation` and `{BASE}/api/get/port_segmentation.json` methods retrieve the port segmentation configuration in JSON format, using the same data structure as the on-the-fly remote command for setting port segmentation [Post][OnTheFly] Set Port Segmentation.

3.17 [Get] Retrieve the port segmentation in msgpack format

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/get/port_segmentation.msgpack` method retrieves the port segmentation configuration in msgpack format, using the same data structure as the on-the-fly remote command for setting port segmentation [Post][OnTheFly] Set Port Segmentation.

3.18 [Get] Retrieve the TC10 State in json Format

This command is supported on the following Technica devices:				
TE-1200	"TE-2003 to TE-2005"			

The `{BASE}/api/port/X/tc10/state` method retrieves the current TC10 state of the specified port. The response contains the current TC10 state for the selected port in json format.

Use this command to monitor the TC10 status for diagnostic or control purposes.

- The field `status` can take the following values:

status value	TC10 status
0	TC10 Disabled
1	Normal
2	Sleep
3	Sleep Failed
4	Sleep Aborted

4 Application HTTP Post Methods

4.1 [Post] Validate configuration

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/validate/config.msgpack` receives a configuration encoded in the msgpack format and:

- Checks if all fields are within the limits. If any field is beyond its limits the configuration is rejected.
- Any missing fields are added and filled with their default value.
- Any field not defined in the device configuration is discarded.

The configuration is sent back corrected or an error indication if the configuration is not valid and cannot be fixed. The sent configuration is neither saved in RAM nor in FLASH. This request can be used to migrate a configuration from an older version to a newer one.

4.2 [Post] Configuration Control

Configuration control requests allow users to modify and securely save the system configuration. If the configuration is password-protected, the password must be provided before sending any data. For more details, refer to the [Post] Password commands chapter.

4.2.1 [Post][Save] Write the whole configuration in a persistent way

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/set/strict/config_persistent.msgpack` method stores the configuration sent in the payload of this request, encoded in msgpack format, into the NVM (Non-Volatile Memory). This ensures the configuration is preserved even after a reset or a power loss. The sent configuration must fully comply with the structure that defines the device configuration (no missing fields, no extra fields).

For the configuration to become effective it is necessary to send a cold reset command (`{BASE}/api/cmd/reset_cold`) or to turn off/on the power supply.

4.2.2 [Post][Save] Write part of the configuration in a persistent way

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/set/partial/config_persistent.msgpack` method stores a subset of the device configuration sent in the payload of this request, encoded in msgpack format, into the NVM (Non-Volatile Memory). This ensures the configuration is preserved even after a reset or a power loss. The sent configuration may contain a subset of the device configuration, parameters not included in the sent data will remain unchanged.

For the configuration to become effective it is necessary to send a cold reset command (`{BASE}/api/cmd/reset_cold`) or to turn off/on the power supply.

4.2.3 [Post][Save] Write a whole configuration in a non-persistent way

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/set/strict/config_non_persistent.msgpack` method stores the device configuration sent in the payload of this request, encoded in msgpack format, into the RAM (A Volatile Memory). Therefore this configuration will be lost in case of a cold reset or a power off/on. The sent configuration must fully comply with the structure used to define the device configuration (no missing fields, no extra fields).

For the configuration to become effective it is necessary to send a warm reset command (`{BASE}/api/cmd/reset_warm`).

4.2.4 [Post][Save] Write a configuration in a non-persistent way

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The `{BASE}/api/set/partial/config_non_persistent.msgpack` method stores the device configuration sent in the payload of this request, encoded in msgpack format, into the RAM (A Volatile Memory). The sent configuration may contain a subset of the device configuration, parameters not included in the sent data will remain unchanged.

For the configuration to become effective it is necessary to send a warm reset command (`{BASE}/api/cmd/reset_warm`).

4.3 [Post] Action commands

Action command requests allow the execution of specific actions on the device, enabling users to trigger targeted operations as needed.

4.3.1 [Post][Action] Save the whole configuration in a persistent way

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/cmd/save` method stores the device configuration, in binary format, into the NVM (Non-Volatile Memory). This ensures the configuration is preserved even after a reset or a power loss.

4.3.2 [Post][Action] Cold reset

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200	TE-1180, TE1181		

The `{BASE}/api/cmd/reset_cold` method discards the current configuration stored in RAM and restarts the device. At boot time, the configuration is taken from the NVM. If the configuration was modified before this request, the boot process may take some time to process the new configuration and save it in a special area to avoid this process on subsequent reboots if there are no changes to the configuration.

The WEB application embedded into the device uses this command along with `{BASE}/api/set/strict/config_persistent.msgpack` to save and apply the configuration. The "SAVE and APPLY" button saves the configuration using `{BASE}/api/set/strict/config_persistent.msgpack` and later sends `{BASE}/cmd/reset_cold` to activate the configuration. To check when the "SAVE and APPLY" has finished the Device WEB application uses the `{BASE}/api/cmd/alive` command to poll when the device is ready again.

4.3.3 [Post][Action] Warm reset

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200		

The `{BASE}/api/cmd/reset_warm` method saves the current configuration in a dedicated RAM area that is retained after reboot and then restarts the device. On boot, the configuration is taken from that RAM, processed, and applied to into the device. The processed configuration is not saved, so it's faster than a cold reset but the configuration is not persistent.

4.3.4 [Post][Action] Alive command

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200		

The `{BASE}/api/cmd/alive` method allows checking if the device can accept HTTP requests. An 8-digit number must be sent along with the alive command request. The alive command will respond with the same number.

4.3.5 [Post][Action] Jump to bootloader mode

This command is supported on the following Technica devices:					
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181		

The `{BASE}/api/cmd/bootloader` method discards the current configuration and jumps to the bootloader program.

4.3.6 [Post][Action] Force sleep mode

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/cmd/sleep` discards the current configuration and jumps to sleep mode. Once in sleep mode, the device can only be woken by wake-up lines or bus CAN activity.

4.3.7 [Post][Action] Clear statistics

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/cmd/clear_statistics` method resets all statistics gathered by the device.

4.4 [Post] On-the-fly commands

On-the-fly commands allow to immediately change some device configuration without having to restart the device and without affecting the rest of the device's functionality. The specified parameters are applied both to the hardware and to the current device configuration stored in RAM. The parameters are not stored in the NVM, so the configuration will be lost after a cold reset or power cycle. A warm reset retains any configuration changes made using On-The-Fly commands. To make these changes persistent, use the [Post][Action] Save the whole configuration in a persistent way to store the configuration in NVM.

4.4.1 [Post][OnTheFly] Address Resolution Table

4.4.1.1 [Post][OnTheFly][ART] Remove all automatic learned entries

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/del_all_learned` method discards all MAC addresses learned by the device so far, so the learning process will start from scratch.

4.4.1.2 [Post][OnTheFly][ART] Remove an entry

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/del_entry.msgpack` removes the specified entry (either learned or statically defined in the configuration). If the entry is static, the entry will be removed from the device configuration too. Within the request the MAC address and VLAN ID must be sent encoded using the msgpack format.

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "mac": [1, 2, 3, 4, 5, 6],
        "vlan_id": 100
}
```

Where:

- The `mac` field contents the MAC address to add in the static list in decimal format
- The `vlan_id` field specifies the vlan id of the frames in decimal format.

In this example, the MAC address "01:02:03:04:05:06" with VLAN 100 is removed from the auto-learned list or the static list from the device configuration.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.1.3 [Post][OnTheFly][ART] Add a static entry

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/add_entry.msgpack` method adds a static entry in the switch and also in the device configuration. Within the request, the MAC address and VLAN ID must be sent encoded using the msgpack format.

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "mac": [1, 2, 3, 4, 5, 6],
        "vlan_id": 100
        "ports": [0, 1]
}
```

Where:

- The `mac` field contents the MAC address to add in the static list in decimal format
- The `vlan_id` field specifies the vlan id of the frames in decimal format.
- The `ports` field specifies the ports for which the specified MAC/vlan frame is to be forwarded.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.1.4 [Post][OnTheFly][ART] Replace an entry

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/at/replace_entry.msgpack` replaces a static entry with a new one. If the entry to be replaced is not found, the new entry will be added.

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "del": {
                "mac": [1, 2, 3, 4, 5, 6],
                "vlan_id": 100
        },
        "add": {
                "mac": [1, 2, 3, 4, 5, 6],
                "vlan_id": 101
                "ports": [0, 1]
        }
}
```

Where:

- The `mac` field contents the MAC address to add in the static list in decimal format
- The `vlan_id` field specifies the vlan id of the frames in decimal format.
- The `ports` field specifies the ports for which the specified MAC/vlan frame is to be forwarded.

This command combines the remove and add methods in a single step. It also allows updating the **`ports`** of a MAC entry if both the **`mac`** address and **`vlan_id`** have the same values in the **`del`** and **`add`** nodes.

msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.2 [Post][OnTheFly] Set port physical layer parameters

This command is supported on the following Technica devices:				
TE-1200	TE-2000 to TE-2006			

The `{BASE}/api/port/X/set.msgpack` method allows enabling or disabling a port and modifying its physical layer parameters, such as master/slave role, speed, mode, and others depending on the port type.

Here, `X` represents the port ID.

The available parameters depend on the port type. Only use parameters that are valid for the port you are configuring.

The available parameters vary by port type. Only include parameters that are applicable to the specific port you are configuring.

The msgpack-encoded data to be included in the POST request body must follow the format shown below:

```
{
        "enable": true,
        "master": false,
        "speed": 1000,
        "mode": 0,
        "plca":
        {
           “node_id”: 0,
           “node_count”: 8,
           “tx_opportuinity_bt”: 32
        },
        "burst":
        {
```

```
            "enable": false,
            "count": 0,
            "timer_bt": 128
        },

        "termination": false,
}
```

Where:

- The field `speed` can take the following values:

speed value	speed
0	AUTO (only for Base-T interface)
10	10Mbps (only for Base-T interface)
100	100Mbps
1000	1Gbps

- The field `mode` can take the following values:

mode value	SFP configuration
0	10GBASE-R interface
1	USXGMII interface
2	SGMII interface
3	1000BASE-X interface
4	2500BASE-X interface
5	5GBASE-R interface

- The field `tx_opportuinity_bt` and `timer_bt` are expressed in 10BASE-T1S bit times, meaning their values increment in steps of 0.1 microseconds (e.g., a value of 32 corresponds to 3,2 µs).

If any of these fields are omitted, the corresponding current value remains unchanged.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.3 [Post][OnTheFly][Ptp] Set time on PTP grandmaster partitions

This command is supported on the following Technica devices:				
TE-2000 to TE-2006				

The `{BASE}/api/ptp/origin_timestamp.msgpack` method allows changing the time transmitted by a gPTP or PTPv2 grandmaster partition.

The msgpack data to send embedded in the POST request shall have the following format

```
[
        {
                "partition": 0,
                "init_time": 1737031944,
                "init_time_ns": 5000,
        },
        {
                "partition": 2,
                "init_time": 946684800,
                "init_time_ns": 0,
        }
]
```

Where:

Data is supplied as an array of objects, with each object representing a different PTP partition. Partitions not included in the array remain unaffected.

- The field `partition` indicates the index of the partition as stored in the configuration.
- The field `init_time` represents the number of seconds elapsed since January 1, 1970, 00:00:00 UTC
- The field `init_time_ns` allows to add an offset in nanoseconds in the init_time value.

If `init_time` or `init_time_ns` are omitted, the corresponding current value remains unchanged.

msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.4 [Post][OnTheFly] Modify MKA key

This command is supported on the following Technica devices:				
TE-2000 to TE-2005				

The `{BASE}/api/mka/X/set_key.msgpack` method allows changing a MKA key.

Where `X` is the position of the key as stored in the configuration (starting from 0). To add a new key, use the last available key index plus 1. For example, if the system has three keys with indices 0, 1, and 2, then to add a new key, set `X` = 3.

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "label": "key label",
        "cak": [01, 02, 03, 04, 05, 06, 07, 08, 09 ,10 ,11 ,12 ,13 ,14 ,15 ,16],
        "ckn": [01, 02, 03]
}
```

Where:

- The field `label` specifies the label assigned to the key.
- The field `cak` defines the CAK key, which must be an array of 16 or 32 bytes encoded in decimal.
- The field `ckn` defines the CKN key, which must be an array of up to 16 bytes encoded in decimal.

If any of these fields are omitted, the corresponding current value remains unchanged.

msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.5 [Post][OnTheFly] Set Port Segmentation

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/port_segmentation/set.msgpack` method enables or disables global port segmentation and allows modifying the port segmentation settings for each port.

The msgpack data to send embedded in the POST request shall have the following format

```
{
        "enable": true,
        "rules": [
                { "in": 0, "out": [1,2,3] },
                { "in": 1, "out": [0,2,3] },
                { "in": 8, "out": [] }
        ]
}
```

Where:

- The `enable` field globally enables or disables port segmentation when set to true or false. If this field is not provided, the current global port segmentation configuration remains unchanged.
- The **`rules`** field array defines the new port segmentation for each port. The **`in`** field specifies the port of ingress, while the **`out`** array lists the ports to which frames entering the port specified in **`in`** are allowed to egress. Ports not included in the rules array (ports where the **`in`** field is not specified) will remain unchanged.

The [Get] Retrieve the port segmentation in json format API returns the port segmentation configuration in a similar format.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.6 [Post][OnTheFly] Set The VLAN configuration per port

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1200			

The `{BASE}/api/vlan/port/X/set.msgpack` method allows modifying the vlan configuration of port **`X`**.

Where `X` is the port id:

The msgpack data to send embedded in the POST request shall have the following format

```
{
    "defaultid": 3,
    "remark": 3,
    "membership": {
        "add": [ 3, 4, 5, 6, 7, 8 ],
        "del": [ 2, 9 ]
    },
    "untagged": {
        "add": [ 3, 4 ],
        "del": [ 10 ]
    },
    "tagged": {
        "add": [ 5, 6 ],
        "del": [ 11 ]
    }
}
```

Where:

- All nodes/parameters are optional
- All values are coded decimal
- The `defaultid` field is the new default VID (VLAN Identifier) to be applied for port **`X`**. Accepted values: from 2 to 4095.
- The `remark` field is the new vlan egress policy for port **`X`**
  Accepted values: from 0 to 3:

remark value	Remark/Egress Tagging Policy
0	Egress the frame as received in the input port
1	Remove the VLAN tag upon frame egress
2	Add the VLAN tag to untagged frames upon egress
3	Custom, uses untagged and tagged fields to define the VLAN remarking

- The `membership` field is the list of VIDs supported by port `X` (or not supported based on port `X` ingress policy)
- The `untagged` field is the list of VIDs where frames with these VIDs should egress port `X` untagged.
- The `tagged` field is the list of VIDs where frames with these VIDs should egress port `X` tagged.
- The `add` field is the list of VIDs to be added to its respective list (parent node)
- The `del` field is the list of VIDs to be deleted from its respective list (parent node)
    - Maximum length of **`add`** and **`del`** arrays: 8 VIDs for each list
    - Accepted values: from 2 to 4095.
    - After processing the membership add/del lists, The values that are not present in the final membership list will be neglected.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.7 [Post][OnTheFly] Advanced Filter

This command is supported on the following Technica devices:				
TE-2000 to TE-2006				

The `{BASE}/api/rule/X/set.msgpack` method allows you to update all fields of the advanced filter rule at index `X`.

`X` represents the index of the rule inside the rules list in the device configurations to be modified and can range from 0 to 31. Since double rules occupy two rule slots, the index `X` is determined as follows:

- For Single Rules and the First Part of a Double Rule:
  `X` = Rule position in the configuration + Number of double rules defined before this rule
- For the Second Part of a Double Rule:
  `X` = Rule position in the configuration + Number of double rules defined before this rule + 1

- All rule parameters can be modified on the fly except the rule type (Single or Double)
- User should provide a valid pair of rule ID and Rule data
- In case the user provides invalid data, a descriptive response will be provided to the user.

The msgpack data to send embedded in the POST request shall have the following format

For a single rule

```
{
    "name": "SingleName",
    "enable": true,
    "match": {
        "ports": [0, 1, 2, 3 ],
        "all": false,
        "qinq": false,
        "data": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 8, 0, 64, 0, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 10, 105, 123, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
        "mask": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 255, 255, 240, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 0, 255, 255, 255, 255, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ]
    },
    "action": {
        "ports": [
            8,
            9
        ],
        "dvp": 1,
        "flowid": 0
    }
```

```
}
```

For the first part of a double rule

```
{
    "name": "DoubleFirstRule",
    "followup": true,
    "enable": true,
    "match": {
        "ports": [ 3, 4, 5, 6, 7 ],
        "all": false,
        "qinq": false,
        "raw": true,
        "data": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 8, 0, 64, 0, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 10, 105, 123, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
        "mask": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 255, 255, 240, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 0, 255, 255, 255, 255, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ]
    },
    "action": {
        "ports": [ 8, 9 ],
        "dvp": 1,
        "flowid": 0
    }
}
```

For a second part of a Double rule

```
{
"match":
        {
        "data": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 8, 0, 64, 0, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 10, 105, 123, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
        "mask": [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 255, 255, 240, 0, 0, 0, 0, 0,
                  0, 0, 0, 0, 0, 0, 255, 255, 255, 255, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ]
         },
}
```

Where:

- All nodes/parameters are optional unless otherwise specified.
- All values are decimal
- The `name` field is the name of the rule
- The `enable` field should be true to enable the rule.
- The `followup` field If true, identifies the first part of a double rule, if false then a single rule.
- The `match` field is a collection of parameters used to match a specific frame
    - The `ports` field is the list of ports that applies this rule
    - The `all` flag indicates that all frames should be captured
    - The `qinq` flag indicates that the captured frames should be QINQ tagged (Double Tagged Frame)
    - The `data` field is the data bytes to be matched with the frame data
    - The `mask` field is the mask for the data field
    - The `action` represents the actions to be performed on the frames captured by this rule.

For further details of the advanced filter rules configurations, please refer to the device configurations.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.8 [Post][OnTheFly] Set system date and time

This command is supported on the following Technica devices:				
TE-1190 to TE-1191				

The `{BASE}/api/set/config/datetime` method sets the system date and time. The date and time shall be transmitted on the POST request encoded as plain text, indicating the number of milliseconds elapsed since January 1, 1970, 00:00:00 UTC. This method is only available when the device is set in PTP or gPTP Grandmaster mode or in standalone mode (the devices do not have a specific PTP or gPTP configuration).

4.4.9 [Post][OnTheFly] Trigger remote I2C operation

This command is supported on the following Technica devices:				
TE-1190				

The `{BASE}/api/cmd/remoteI2C` method triggers an I2C operation to be executed by the CM Serdes GMSL2/3 on the GMSL control I2C bus. Within the request, the I2C channel, operation, slave device address, register address and data must be sent encoded using the msgpack format.

The msgpack data to send embedded in the POST request shall have the following format:

```
{
        "i2c_channel": 0
        "i2c_opcode": 1,
        "i2c_device_addr": 196,
        "i2c_register_addr": 16,
        "i2c_value": 33

}
```

Where:

- The `i2c_channel` contents the I2C channel where the operation will be triggered. The I2C channel is equal to the GMSL link (where the I2C operation is required) minus 1. In the example above, the operation will be triggered to the GMSL link 1.
- The `i2c_opcode` field contents whether the CM shall trigger an I2C read request (`i2c_opcode = 0`) or an I2C write request (`i2c_opcode = 1`)
- The `i2c_device_addr` field contents the address of the I2C slave device
- The `i2c_register_addr` field contents the address the register to be read or written
- The `i2c_value` field contents the data to be written to the register

In this example, the CM will trigger an I2C operation (on the GMSL link 1) to write 0x21 to the register 0x10 of an I2C slave device with address 0xC4.

msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.10 [Post][OnTheFly] Start Frame Generator

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/generator/start` method starts the frame generator on port `X` using the configuration provided in the POST request, encoded in msgpack format. Where `X` is the port id.

The frame generator allows the device to transmit a custom pattern of Ethernet frames. The configuration must be valid, otherwise an error message is returned.

The msgpack data to send embedded in the POST request shall have the following format (shown here in JSON for clarity):

```
{
```

```
  "continuous": true,
  "num_packets": 100,
  "errors": 0,
  "payload": 0,
  "packet_size": 1,
  "ipg": 12
}
```

Where:

- `continuous` (bool):
  - true: generates packets indefinitely.
  - false: generates a finite number of packets (`num_packets` must be set).
- `num_packets` (integer, 1–65535): Number of packets to transmit (used only when continuous is false).
- `errors` (bool): Intentionally send with CRC and symbol errors.
  - true: enables corrupted frames.
  - false: normal operation
- `payload` (integer): Defines the payload content:
  - 0 → Pseudo-random
  - 1 → Repeated pattern 0xA5, 0x5A
  - 2 → All 0x00
  - 3 → All 0xFF
- `packet_size` (integer): Defines the size of each packet:
  - 0 → 64 bytes
  - 1 → 1518 bytes
- `ipg` (integer, 1–255): Inter-packet gap, in bytes.

Transmission status can be monitored through the status API:
`{BASE}/api/get/status.json → status.ports.main[X].transmission.generator`
`{BASE}/api/get/status.json → status.ports.base_t1[X].transmission.generator`

msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.11 [Post][OnTheFly] Stop Frame Generator

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/generator/stop` method stops the frame generator running on port `X`. Where `X` is the port id.

This command is used to manually halt a transmission that was started in continuous mode using the corresponding `generator/start` method. It has no effect if the generator is not active or if it was configured in non-continuous mode and already finished.

No data is required in the POST request body. Only the port index `X` must be included in the URL.

Transmission status can be monitored through the status API:

`{BASE}/api/get/status.json → status.ports.main[X].transmission.generator`

`{BASE}/api/get/status.json → status.ports.base_t1[X].transmission.generator`

4.4.12 [Post][OnTheFly] Start IEEE Test Mode

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/test_mode/start` method starts an IEEE Test Mode operation on port `X`. It is used to trigger standardized transmission patterns defined by the IEEE for PHY compliance and validation testing.

The POST request must include a valid msgpack-encoded configuration specifying the test mode to apply.

```
{
  "mode": 1
}
```

Where:

- `mode` (integer): selects the IEEE test mode:
  - 1 → Test mode 1 (repeatedly transmit DME encoded ones)
  - 2 → Test mode 2 (repeatedly positive/negative differential voltages for 1.6 us each)
  - 3 → Test mode 3 (pseudo-random sequence of positive and negative voltage levels)
  - 4 → Test mode 4 (The transmitter is high impedance during the 'I' symbol.)

Only one test mode can be active at a time per port. An error will be returned if a mode is already active.

Transmission status can be monitored through the status API:
`{BASE}/api/get/status.json → status.ports.main[X].transmission.test.mode`
`{BASE}/api/get/status.json → status.ports.base_t1[X].transmission.test_mode`



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.13 [Post][OnTheFly] Stop IEEE Test Mode

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/test_mode/stop` method stops the IEEE Test Mode currently active on port `X`. It can be used to halt any ongoing IEEE PHY compliance transmission started using the corresponding `test_mode/start` command.

No data is required in the POST request body. Only the port index `X` must be set in the URL.

Transmission status can be monitored through the status API:
`{BASE}/api/get/status.json → status.ports.main[X].transmission.test.mode`
`{BASE}/api/get/status.json → status.ports.base_t1[X].transmission.test_mode`



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request

must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.14 [Post][OnTheFly] Cable Diagnostics

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/diagnostics` method runs a cable diagnostic test on the specified 10BASE-T1S port `X`.

This feature triggers a low-level PHY test to determine the physical status of the cable connected to the port. It can detect conditions like short, open, or normal connectivity.

No parameters are required in the POST body. Only the port index X must be set in the URL. The command only applies to `main` 10BASE-T1S ports. If the selected port is not valid, an error message is returned.

The response is a msgpack-encoded object containing the result of the test. The typical structure is as follows:

```
{
  "result": 0,
  "distance": 0
}
```

Where:

- `result` (integer):
  - 0 → NORMAL cable
  - 1 → SHORT cable
  - 2 → OPEN cable
  - 3 → FAILED test (invalid)
- `distance` (integer): Reserved for future use (always 0 for now).

The response may take some time (up to a few hundred milliseconds) to complete. The request blocks until the result is ready.



msgpack is a binary format and cannot be represented in a text document like this. To represent the format in this specification the json format is used, but the data sent in the POST request must be encoded in msgpack format. Refer to the Python snippet scripts chapter for more details.

4.4.15 [Post][OnTheFly] Set a TC10 Local Sleep Event

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/tc10/local_sleep` method triggers a TC10 Local Sleep event on the specified port `X`.

Use this command to command the PHY of the port to transition to TC10 Sleep mode locally.

No additional parameters are required in the POST request body. Only the port index `X` must be set in the URL.

4.4.16 [Post][Notedly] Set a TC10 Local Wake Event

This command is supported on the following Technica devices:				
TE-1200				

The `{BASE}/api/port/X/tc10/local_wake` method triggers a TC10 Local Wake event on the specified port `X`.

Use this command to command the PHY of the port to transition from TC10 Sleep to Normal mode locally.

No additional parameters are required in the POST request body. Only the port index `X` must be set in the URL.

4.4.17 [Post][OnTheFly] Set a TC10 Remote Sleep (LPS) Event

This command is supported on the following Technica devices:				
TE-1200	TE-2003 to TE-2005			

The `{BASE}/api/port/X/tc10/lps` method triggers a Remote Sleep (LPS) event via TC10 on the specified port `X`.

Use this to request a link to transitioning to sleep via TC10 signaling.

No additional parameters are required in the POST request body. Only the port index `X` must be set in the URL.

4.4.18 [Post][OnTheFly] Set a TC10 Remote Wake-Up (WUP) Event

This command is supported on the following Technica devices:				
TE-1200	TE-2003 to TE-2005			

The `{BASE}/api/port/X/tc10/wup` method triggers a Remote Wake-Up (WUP) event via TC10 on the specified port `X`.

Use this to request a remote device(s) to wake up via TC10 signaling.

No additional parameters are required in the POST request body. Only the port index `X` must be set in the URL.

4.5 [Post] Password commands

To enhance security, device configuration settings can be protected with a password. When password protection is enabled, changes to the configuration are only possible if the correct password is provided.

For security reasons:

- The password is neither stored nor transmitted in plain text.
- The system never sends the password directly.

- A nonce-based authentication mechanism is used to prevent replay attacks, ensuring that each authentication request is unique.

This ensures secure access to device settings while preventing unauthorized modifications.

Once the configuration is protected by password, the following remote API needs to include the password encoded in the POST request:

- [Post][Save] Write the whole configuration in a persistent way
- [Post][Save] Write part of the configuration in a persistent way
- [Post][Save] Write a whole configuration in a non-persistent way
- [Post][Save] Write a configuration in a non-persistent way

When sending a password in a POST request, it must be encoded as a 32-byte sequence and included at the beginning of the request data.

```
Header Added: SHA-256(NONCE bitwiseOR SHA-256(current password encoded in UTF-8))
```

For more information on password protection and authentication, refer to the commands listed in this section.

4.5.1 [Post][Password] set password

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The **`{BASE}/api/password/set`** command sets a new password. The new password must be encoded as a 64-byte sequence and included in the POST request.

Password Encoding Process

- **`Bytes 0-31: SHA-256(NONCE bitwiseOR SHA-256(current password encoded in UTF-8))`**
  This ensures that the request is unique and prevents replay attacks. If no password has been set use "no password" as the current password. NONCE is a 32 byte number that must be retrieved using the [Post][Password] nonce command.
- **`Bytes 32-63: SHA-256(current password encoded in UTF-8) bitwiseXOR SHA-256(new password encoded in UTF-8)`**
  This securely encodes the transition from the current password to the new one. If no password has been set use "no password" as the current password.

Once the command is executed, the password will be updated after the next reset or power-up.

4.5.2 [Post][Password] clear password

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The **`{BASE}/api/password/clear`** command removes the password protection. The current password must be encoded as a 32-byte sequence and included in the POST request.

Password Encoding Process

- **`Bytes 0-31: SHA-256(NONCE bitwiseOR SHA-256(current password encoded in UTF-8))`**
  This ensures that the request is unique and prevents replay attacks. If no password has been set use "no password" as the current password. NONCE is a 32 byte number that must be retrieved using the [Post][Password] nonce command.

Once the command is executed, the password will be removed after the next reset or power-up.

4.5.3 [Post][Password] nonce command

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

The **{BASE}/api/password/nonce** command returns the nonce key used during the authentication process. The key is a 32-bytes sequence transmitted directly in the post request answer.

4.5.4 [Post][Password] protected command

This command is supported on the following Technica devices:				
TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181	

To check whether the device configuration is protected by a password, use the **{BASE}/api/password/protect** command. This POST request returns a single-byte response:

1 → The configuration is **protected** by a password.
0 → The configuration is **not protected** by a password.

5 Bootloader API

When the device is in bootloader mode, only the remote API commands listed in this section are available. To enter bootloader mode, use the remote command described in section [Post][Action] Jump to bootloader mode.

5.1 [Post] Return back to main application

This command is supported on the following Technica devices:					
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177	TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181

The `{BASE}/reset` method resets the device and jumps to application (if available).

5.2 [Post] Retrieve the system information in json format

This command is supported on the following Technica devices:					
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177	TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181

The `{BASE}/get_version` command returns a summary of the device's full hardware details and currently installed software. The information is returned in JSON format as shown below:

```
{
  "hardware_info": "TE-XXXX [XXX] HW v1.X",
  "serial_number": "2000100025010001",
  "maintenance": "01/01/2030",
  "fbl_version": "R001.000.000 01/01/2025 00:00:00",
  "app_version": "R001.000.000 01/01/2025 00:00:00",
  "sys_version": "R.001.000"
}
```

5.3 [Post] Retrieve the status of the firmware update in json format

This command is supported on the following Technica devices:					
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177	TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200	TE-1180, TE1181

The `{BASE}/firmware_progress` command provides a summary of the firmware update process, including the device module being updated, the operation in progress, the completion percentage, and the result.

```
{
  "device": "X",
  "operation": "Y",
  "percentage": "0",
  "result": "Z",
}
```

- The `"device"` field values are specific to each equipment, as each equipment has different devices to be updated.
- The `"operation"` field can take the following values:
  - `"preparing"`
  - `"erasing"`
  - `"programming"`
  - `"validating"`
  - `"update_confirmation"`
  - `"completed"`

- The `"percentage"` field is a string that represents the percentage of completion of the operation currently running.
- The `"result"` field can take the following values:
  - `"FW_NOT_STARTED"`
  - `"FW_COMPLETED"`
  - `"FW_ONGOING"`
  - `"FW_ERR_ALREADY_STARTED"`
  - `"FW_ERR_DEVICE_PREPARE"`
  - `"FW_ERR_DEVICE_ERASE"`
  - `"FW_ERR_DEVICE_PROGRAM"`
  - `"FW_ERR_DEVICE_VALIDATE"`
  - `"FW_ERR_UNRECOGNISED_FILE_FORMAT"`
  - `"FW_ERR_INPUT_TRUNCATED"`
  - `"FW_ERR_INPUT_EXTRA_BYTES"`
  - `"FW_ERR_ZLIB_INFLATE"`
  - `"FW_ERR_PRODUCT_MISMATCH"`
  - `"FW_ERR_INCONSISTENCY_ERROR"`
  - `"FW_ERR_LICENSE_FORBIDDEN"`
  - `"FW_ERR_INVALID_ELEMENT_FORMAT"`
  - `"FW_ERR_INVALID_TARGET"`
  - `"FW_ERR_SIGNATURE_INVALID"`
  - `"FW_ERR_NO_KEY"`
  - `"FW_ERR_LICENSE_FILE_REJECTED"`
  - `"FW_ERR_FIRMWARE_TOO_OLD"`
  - `"FW_ERR_BOOTLOADER_TOO_OLD"`
  - `"FW_ERR_INTERNAL_ERROR"`
  - `"FW_ERR_DEVICE_PARALLEL_START"`
  - `"FW_ERR_DEVICE_LOCAL_CONFIRM"`
  - `"FW_ERR_DEVICE_GLOBAL_CONFIRM"`

5.3.1 [POST][Bootloader] Update firmware or license file

This command is supported on the following Technica devices:				
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177	TE-2000 to TE-2006	TE-1190 to TE-1191	TE-1200

The `{BASE}/firmware_update` method updates the device firmware or license. The "tefw" file containing the new firmware or license is sent as raw binary data within the post request. Upon a successful update, the device undergoes a reset, and if possible, jumps to the new application.

5.3.2 [POST][Bootloader] Save firmware or license file without reset

This command is supported on the following Technica devices:				
TE-1170, TE-1171, TE-1173, TE-1175	TE-1176 to TE-1177	TE-2000 to TE-2006		

The `{BASE}/firmware_update_noreset` method updates the device firmware or license. The "tefw" file containing the new firmware or license is sent as raw binary data within the post request. Once the request is finished, the device remains in bootloader mode.

5.4 [Post] Restore configuration to factory default

This command is supported on the following Technica devices:				
TE-1190 to TE-1191				

The `{BASE}/factory_reset` method restores the device to its factory settings, erasing all configurations, including network settings.

6 Examples of use

The remote API is based on HTTP requests, so any tool that allows sending GET and POST requests can be used. GET requests can be generated directly from a web browser, so no further tools are needed to retrieve the data available through the remote command.

This section provides examples for accessing the remote API with standard tools such as the curl command and python scripts.

6.1 Curl

The curl command is a tool used to transfer data from or to a server using various network protocols, including HTTP, HTTPS, FTP, and others. It is commonly used in command-line interfaces and scripts for tasks such as downloading files, sending data to a server, and testing APIs. The most relevant options for dealing with the Technica remote API are:

- `-v` "optional" verbose mode, additional information about the request will be shown.
- `-o` specify the file where the data read from the get or post http request will be saved.
- `-s` "optional" silent mode, no statistics are shown.
- `-d` provides the associated data in the body of the HTTP request. Force a POST request.
- `--data-binary @file_name` send the file content exactly with no extra processing. When `--data-binary` is used curl will use a POST request.

6.1.1 [Curl] Read all current configuration data in msgpack and save it in a file

```
curl http://10.0.0.200/api/get/config.msgpack -o cfg.msgpack -s
```

6.1.2 [Curl] Read all current configuration data in Json format and save it in a file

```
curl http://10.0.0.200/api/get/config.json -o cfg.json -s
```

6.1.3 [Curl] Read part of current configuration data in Json format and save it in a file

For instance all parameters of the BaseT-1 port

```
curl http://10.0.0.200/api/get/config/ports/main/0 -o cfg.json -s
```

6.1.4 [Curl] Read all current status data in Json format and save it in a file

```
curl http://10.0.0.200/api/get/status.json -o status.json -s
```

6.1.5 [Curl] Check is HTTP requests can be accepted

The device shall respond with the same number sent into the POST request.

```
curl -d "01234567" http://10.0.0.200/api/cmd/alive -s
```

6.1.6 [Curl] Send the complete configuration and save it in the device's NVM

The configuration will be effective once a power off/on cycle or a cold reset command is sent

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/set/strict/config_persistent.msgpack -s
```

Reset command to force a cold reset of the device and set the configuration stored in the Non-Volatile Memory of the device.

```
curl -d "" http://10.0.0.200/api/cmd/reset_cold -v
```

6.1.7 [Curl] Send the complete configuration and save it in the device's RAM

The configuration will be effective after a warm reset command is sent. This configuration will be lost in a power off/on cycle.

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/set/strict/config_non_persistent.msgpack -s
```

Reset command to force a warm reset of the device and set the current configuration stored in RAM.

```
curl -d "" http://10.0.0.200/api/cmd/reset_warm -v
```

6.1.8 [Curl] Validate a configuration

Check if the sent configuration is correct, if not, try to fix it if possible. The configuration is not stored in RAM or NVM.

```
curl --data-binary @cfg.msgpack \
http://10.0.0.200/api/validate/config.msgpack -o validated.msgpack -s
```

6.1.9 [Curl] Jump to bootloader

Jump to bootloader from the main application.

```
curl -d "" http://10.0.0.200/api/cmd/bootloader
```

6.1.10 [Curl] Update the firmware once in bootloader

In bootloader mode, update a new firmware and stay in bootloader.

```
curl --data-binary @firmware_full_signed.tefw http://10.0.0.200/firmware_update_noreset -o -
```

6.2 Python snippet scripts

To use the following python script, it is necessary to install the msgpack, hashlib, and requests modules.

```python
import msgpack
import requests

IP_DST = '10.0.0.200'

URL_DEFAULT =        'http://' + IP_DST + '/api/get/default_config.msgpack'
URL_VALIDATE =       'http://' + IP_DST + '/api/validate/config.msgpack'
URL_UPLOAD =         'http://' + IP_DST + '/api/set/strict/config_persistent.msgpack'
URL_RESET_COLD =     'http://' + IP_DST + '/api/cmd/reset_cold'

print("read default configuration")
response = requests.get(url=URL_DEFAULT)
msgpack_data = response.content
json_data = msgpack.unpackb(msgpack_data)
print(json_data)
input("Press Enter to continue...")

print("validate the configuration read previously")
response = requests.post(url=URL_VALIDATE, data=msgpack_data)
msgpack_data = response.content
json_data = msgpack.unpackb(msgpack_data)
print(json_data)
input("Press Enter to continue...")

print("write back the configuration")
response = requests.post(url=URL_UPLOAD, data=msgpack_data)
print("write strict: ", response.text)
input("Press Enter to continue...")

print("A reset is necessary for the new configuration to be activated")
response = requests.post(url=URL_RESET_COLD)
print("reset strict: ", response.text)
```

Figure 1. Python snippet to read, validate and write a configuration

```python
import hashlib
import requests

IP_DST = '10.0.0.200'

URL_RESET_COLD =     'http://' + IP_DST + '/api/cmd/reset_cold'
URL_PASSWORD_NONCE = 'http://' + IP_DST + '/api/password/nonce'
URL_PASSWORD_SET =   'http://' + IP_DST + '/api/password/set'


print("Set a new password")
old_password = "OldPassWord"
new_password = "NewPassWord"
old_password_hash = hashlib.sha256(old_password.encode("utf-8")).digest()
new_password_hash = hashlib.sha256(new_password.encode("utf-8")).digest()

nonce = requests.post(url=URL_PASSWORD_NONCE).content
encoded_old_password = hashlib.sha256(bytes([x | y for x, y in zip(nonce, old_password_hash)])).digest()
encoded_new_password = bytes([x ^ y for x, y in zip(new_password_hash, old_password_hash)])
encoded_data = encoded_old_password + encoded_new_password
requests.post(url=URL_PASSWORD_SET, data=encoded_data)
print("A reset is necessary for the new password to be updated")
response = requests.post(url=URL_RESET_COLD)
input("Press Enter to continue...")
```

Figure 2. Python snippet to change the password. If old_password is set to "no password", the same code can be used to set a password in an unprotected device.

7 Remote API reference code

In order to exemplify how accessing the remote API can be automated, Technica provides an example python applications which can be used to automate the process of uploading configuration, firmware and license files.

The example python applications contain the following files:

- WebInterface.py: This file contains the base class to interact with the generic Remote API of a Technica device. This class provides methods for uploading configuration files, firmware/license files, resetting the device, and setting the device to bootloader mode. It uses a requests session for persistent connections and handles retries and timeouts for robustness.
- CMInterface.py: This file contains a specialized class for interacting with the Capture Module web interface, overriding its attributes used to match the Capture Module remotes API URLs and its methods to change the device configuration and set the device in bootloader mode. This file can be imported into your Python applications to automate the interaction with our device.
- NIInterface.py: This file contains a specialized class for interacting with the 10BASE-T1S NI web interface, overriding its attributes used to match the 10BASE-T1S NI remotes API URLs. This file can be imported into your Python applications to automate the interaction with our device.
- EESInterface.py: This file contains a specialized class for interacting with the EES web interface, overriding its attributes used to match the EES remotes API URLs. This file can be imported into your Python applications to automate the interaction with our device.
- ApplicationExample.py: A script interacting with Technica devices (Capture Module, Enhanced Ethernet Switch, or 10BASE-T1S Network Interfacer) to change their configuration and update its firmware. This script allows the user to select the device type and then perform actions such as uploading configuration files, firmware, or licenses.

The previous files are provided as sample applications, they can be modified to adapt to new software releases or configuration files. These files are included in a compressed file embedded within this document.

7.1 Requirements

The Remote API python example scripts should work with any active version of python & pip, although it has been tested with 3.10.12. Visit https://www.python.org to download the python interpreter.

In order to execute the script, it is necessary to install the following libraries (for instance using pip):

- msgpack using pip: `pip install msgpack`
- requests using pip: `pip install requests`
- requests_toolbelt using pip: `pip install requests_toolbelt`

The Remote API python example scripts should be OS-agnostic and compatible with both Windows and Linux OS, however it has only been tested with a Windows system.

7.2 WebInterface base class definition

The WebInterface.py file contains the definition of the WebInterface base class to interact with Technica devices using the Remote API. This class serves as the baseline for device specific classes, integrating the APIs used by most of Technica devices. This file consists of four parts:

```
import requests
import msgpack
from requests_toolbelt import SourceAddressAdapter

import datetime
import time
import json
import os
```

```
class WebInterface:
    def __init__(self, device_ip):
        self.session = None
        self.device_ip = device_ip
        self.url_config_upload = "/api/set/strict/config_persistent.msgpack"
        self.url_firmware_upload = "/firmware_update"
        self.url_reset = "/api/cmd/reset_cold"
        self.url_bootloader = "/api/cmd/bootloader"
        self.url_alive = "/api/cmd/alive"
```

```
    def upload_configuration_file(self, file_path, network_adapter, timeout=30, retries=8,
                                  alive_timeout=10):
    def upload_firmware_or_license_file(self, file_path, network_adapter, timeout=10, retries=2):
    def set_bootloader(self, network_adapter, timeout=0.2, retries=10):
```

```
    def requests_send(self, ip_adapter, dest_ip, url, data = None, timeout=0.2, retries=400, session=None,
                      type = "POST"):
```

Figure 3. Structure of WebInterface.py

- In orange, the required python module imports.
- In blue, the class initialization function and its default attributes
- In purple, the functions which interact with the device to change the device configuration, set it into bootloader mode and upload its firmware or license
- In green, the auxiliary functions.

The WebInterface class is defined with an initialized with the device IP and the Remote API URLs as attributes:

```
class WebInterface:
    def __init__(self, device_ip):
        self.session = None
        self.device_ip = device_ip
        self.url_config_upload = "/api/set/strict/config_persistent.msgpack"
        self.url_firmware_upload = "/firmware_update"
        self.url_reset = "/api/cmd/reset_cold"
        self.url_bootloader = "/api/cmd/bootloader"
        self.url_alive = "/api/cmd/alive"
```

Figure 4. Initialization method for WebInterface Class

The WebInterface class provides the following methods to interact with the Technica Devices Remote API:

- The **`upload_configuration_file(…)`** function oversees uploading a configuration file (encoded in msgpack format) to the device and resetting the device afterwards to apply the requested configuration.
- The **`upload_firmware_or_license_file(…)`** function oversees uploading the firmware or license file to the device. This function will only work if the devices has been previously set into bootloader mode
- The **`set_bootloader(…)`** function oversees sending the device to bootloader.
- The **`requests_send(…)`** function oversees communicating with the device via Ethernet by sending HTTP POST/GET requests to the given url.

7.3 Device-Specific class definition

The example Remote API python scripts package contain a specific file, with its own class definition, for each device family. Each class overrides the required attributes and methods to adapt the generic Remote API to

- CMInterface:
  - URLs for accessing the remote API are overridden.
  - The **`upload_configuration_file(…)`** function is overridden to upload the configuration in binary format (".cfg" binary configuration file) and pool the status of the device to detect when it is available.
  - in charge of uploading a configuration file (encoded in msgpack format) to the device and resetting the device afterwards to apply the requested configuration.
  - The **`set_bootloader(…)`** function is overridden, adapting the HTTP request to set the device in bootloader mode
- NIInterface:
  - URLs for accessing the remote API are overridden.
- EESInterface:
  - URLs for accessing the remote API are overridden.

7.4 Example script application definition

Within the example Remote API python scripts package, an example application script is delivered, allowing a user to interact with the device using a command-line interface. This file shall be executed using python: "python ApplicationExample.py".

When executing this file, a user must first introduce the following information to properly initialize the Remote API interface:

- Network Adapter IP: the IP address of the network adapter that connects local PC with the device.
- Device IP: the device IP address
- Device Type: Capture Module, EES or 10BASE-T1S Network Interfacer

```
#Network configuration
network_adapter_ip = input('Introduce the IP of the network adapter with the format "X.X.X.X"\n')
device_ip = input('Introduce the IP of your device with the format "X.X.X.X"\n')

#Defined Files
print("-----------------------------------------------------------------")
print("\n-----Technica device configuration and Upload Python script Example-----\n")

print('Select the number your device, "q" for quit')
print(' 1 --> Capture Module (CAN Combo, LIN Combo, Ethernet Combo, 100High, 1000High)')
print(' 2 --> Enhanced Ethernet Switch (EES)')
print(' 3 --> Network Interfacer 10BASE-T1S')
key_resp = input()

if key_resp == '1':
    print("")
    print('Device Capture Module selected')
    device = CMInterface(device_ip)

elif key_resp == '2':
    print("")
    print('Enhanced Ethernet Switch selected')
    device = EESInterface(device_ip)

elif key_resp == '3':
    print("")
    print('Network Interfacer 10BASE-T1S selected')
    device = NIInterface(device_ip)
```

Figure 5. Main of CM_Example_Script.py

```
    while run:
        print('Select the number of your  option, "q" for quit')
        print(' 1 --> Upload Config File to the device')
        print(' 2 --> Upload Firmware to the device')
        print(' 3 --> Upload License File to the device')
        key_resp = input()
        print("")
        print("\n----------------------------------------------------------------\n")

        if key_resp == '1':
            print("")
            cfg_file = input('Introduce configuration file name in the following format:'
                             'custom_config.cfg or custom_config.json\n')
            print(f'Introduced configuration file name: {cfg_file}')
            result = device.upload_configuration_file(cfg_file, network_adapter_ip)

        elif key_resp == '2':
            tefw_file = input('Introduce firmware file name in the following format:'
                              'firmware_full_signed.tefw\n')
            print(f'Introduced firmware file name: {tefw_file}')
            print("Setting the device in bootloader state")
            result = device.set_bootloader(network_adapter_ip)
            time.sleep(3)
            print("")
            print('Uploading tefw will take more than 1 minute')
            result = device.upload_firmware_or_license_file(tefw_file, network_adapter_ip,200,1)

        elif key_resp == '3':
            telc_file = input('Introduce license file name in the following format:'
                              '11713302221400047.telc\n')
            print(f'Introduced license file name: {telc_file}')
            print("Setting the device in bootloader state")
            result = device.set_bootloader(network_adapter_ip)
            time.sleep(7)
            print("")
            result = device.upload_firmware_or_license_file(telc_file, network_adapter_ip,10,3)
```

Figure 6. Main of Example_Script.py

Once initialized the script provides the following options:

1- Upload Configuration File to the device: The configuration file indicated will be uploaded to the device
2- Upload Firmware to the device: The device will be set to bootloader mode and the. tefw firmware will be uploaded to the device.
3- Upload License File to the device: The device will be set to bootloader mode and the. telc license will be uploaded to the device.

The following image shows the execution of the sample application script to change the configuration of an EES:

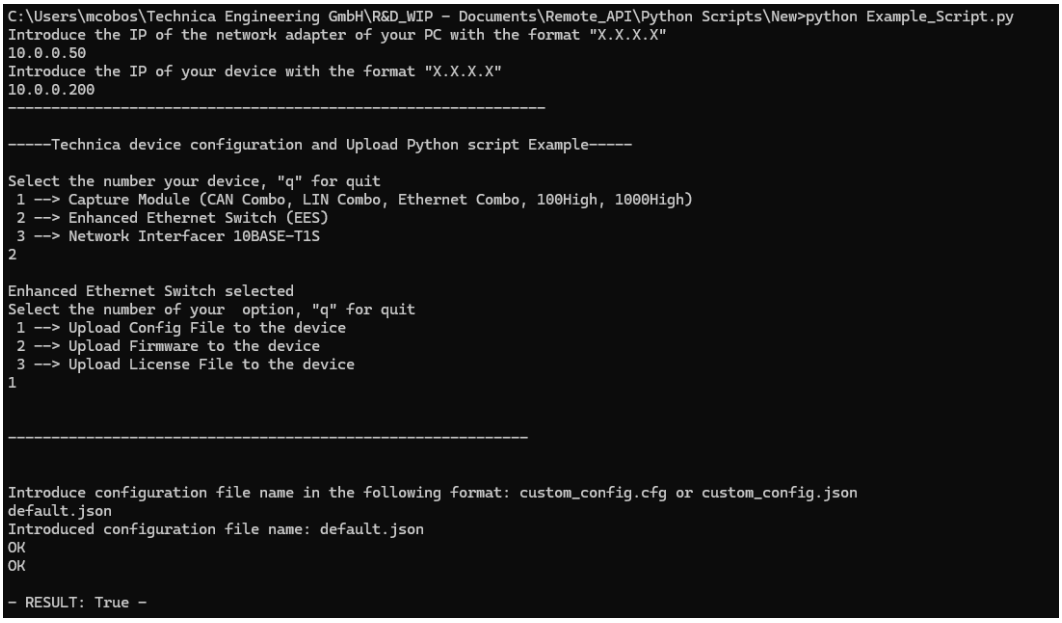


Figure 7. Example_Script.py execution

8 Document Changelog

Version	Date	Change description
1.0	17/04/2025	Initial Release
1.1	13/06/2025	Added new endpoints: • [Post][OnTheFly] Set a TC10 Local Sleep Event • [Post][OnTheFly] Set a TC10 Local Wake Event • [Post][OnTheFly] Set a TC10 Remote Wake-Up (WUP) Event • [Get] Retrieve the TC10 State in JSON Format Expanded endpoints to support TE-1200: • [Post][OnTheFly] Set port physical layer parameters Fixed some typos on following endpoints: • [Post][Action] Jump to bootloader mode
1.2	08/07/2025	Removed endpoint: • [Get] Retrieve the LLDP neighbor list in msgpack format Added endpoint: • [Get] Retrieve the LLDP interfaces list in json format
1.3	13/11/2025	Included TE-2003 to TE-2005 in the following TC10 end points: • [Get] Retrieve the TC10 State in json Format • [Post][OnTheFly] Set a TC10 Remote Sleep (LPS) Event • [Post][OnTheFly] Set a TC10 Remote Wake-Up (WUP) Event
1.4	01/04/2026	Included TE-1200 in the following end points: • [Get] Retrieve the Address Resolution Table in json format • [Get] Retrieve the port segmentation in json format • [Get] Retrieve the port segmentation in msgpack format • [Post][Action] Save the whole configuration in a persistent way • [Post][OnTheFly] Set Port Segmentation • [Post][OnTheFly] Set The VLAN configuration per port Added extra information to retrieve the status of the following end points: • [Post][OnTheFly] Start Frame Generator • [Post][OnTheFly] Stop Frame Generator • [Post][OnTheFly] Start IEEE Test Mode • [Post][OnTheFly] Stop IEEE Test Mode

IMPRINT

Technica Engineering GmbH
Leopoldstraße 236
D-80807 München

Contact
Tel.: +49 89/200 07 24-10
Fax: +49 89/200 07 24-30
E-Mail: info@technica-engineering.de
Website: www.technica-engineering.de

Information in this manual may change without notice. No part of this manual may be reproduced or distributed in any form without written permission from Technica Engineering GmbH. All content is protected by copyright law.

© Copyright 2025, Technica Engineering GmbH. All rights reserved